

Technical Report Summary: RedLine Stealer

Author: Svetoslav Angelov

- **Report ID:** 2026-05-11-RED-02
- **Classification:** TLP:CLEAR
- **Subject:** Manual Deobfuscation and Reflective Loading Analysis of RedLine Stealer

Executive Summary

A detailed static and dynamic analysis was conducted on a suspected RedLine Stealer payload. The malware utilises a heavily obfuscated .NET loader protected by ConfuserEx to conceal its core malicious functionality. Through manual memory carving and dynamic debugging, the analysis bypassed the initial packing layers to reveal a sophisticated reflective loading mechanism. The malware operates entirely within memory, extracting and executing its final payload via **System.Reflection.Assembly.LoadModule** without writing the core stealer to disk.

List of Affected Entities

- **Internal Source (Assigned):** Isolated Malware Analysis Laboratory (Windows 10 x64, Flare-VM)
- **Target Identity:** Local Workstation Host (Execution environment for redline.exe)
- **Attacker IP:** Encrypted within memory heap (Historically associated with 198.46.86.63)
- **Identity:** redline.exe (Initial Dropper/Loader), koi (Reflective Stealer Module)
- **External Actor:** Unknown Commodity Malware Operator (RedLine MaaS Affiliate)

Technical Findings & Analysis Steps

1. Static Analysis (Initial Assessment)

- **Tooling:** Detect It Easy (DIE), dnSpy.
- **Findings:** The initial executable was identified as a 32-bit .NET assembly. DIE detected the presence of the Confuser(1.X) protector. Static analysis in dnSpy revealed heavily obfuscated class names and namespaces, notably a namespace named **Steainings** containing a **LzmaDecoder** class. The presence of LZMA indicated that the file contained compressed, embedded resources. No clear Command and Control (C2) configurations were visible in the static code due to the packing.

2. Dynamic Analysis & Memory Carving

- **Tooling:** x32dbg, ScyllaHide (Stealth Injection profile).
- **Findings:** The malware was detonated under x32dbg to monitor memory allocations during execution. The initial goal was to intercept the unpacked payload after the **VirtualAlloc** API call.
- **Execution:** Automated dumping attempts using Scylla failed due to the .NET architecture lacking a traditional Import Address Table (IAT) for the payload. The analysis pivoted to manual memory map inspection.
- **Extraction:** By tracking memory segments with Execute-Read-Write (ERW) protections, a private memory region at address **00320000** (Size: **00008000**) was identified containing an MZ executable header. This region was manually carved and dumped to disk (*unpacked_payload.exe*) for further static review.

3. Payload Execution Flow (Reflective Injection)

- **Tooling:** dnSpy.
- **Findings:** The carved memory dump was loaded back into dnSpy. While standard namespaces were not fully populated due to memory alignment shifts, the core execution mechanism was exposed. The decompiled code revealed a massive encrypted **uint[]** array containing 25,827 elements.

- **Execution:** The malware utilises the **LzmaDecoder** to decrypt this array into a byte array (**array2**). It then dynamically loads this decrypted byte array directly into memory as a hidden module using the internal name "**koi**" via the **executingAssembly.LoadModule("koi", array2)** instruction. Finally, the malware executes the payload using **MethodBase.Invoke**.
- **Conclusion:** The configuration strings, including the specific C2 IP and net.tcp communication protocols, are dynamically decrypted into the heap only milliseconds prior to execution, successfully evading static detection and basic memory string scans.

Technical Resources

- **Detection Engineering:** [View the custom YARA rule here](#)
- **IoC Publication:** [Access the full list of IoCs on AlienVault OTX](#)